Lecture1: Introduction

1. What is Parallel Programming?

- Using multiple computing resources to solve a problem **faster** and **efficiently**.
- Used for problems requiring **high computational power** for short periods.
- HPC systems range from small workstations to supercomputers.

2. Brief History of Computing

Serial Computing

1. Each problem is broken into a **discrete series of instructions.**
2. Instructions are executed **sequentially** on a **single CPU.**
3. Only one instruction executes at a time → **slow** for complex problems.

Parallel Computing

- **simultaneous** use of multiple compute resources to solve a computational problem.

1. Each problem is broken into **discrete parts solved concurrently.**
2. Each part is broken down to a **series of instructions.**
3. Instructions execute **simultaneously** on **different CPUs.**

- Unified control mechanism manages the process.
- **Benefits:**
  - **saves time & cost**
  - **solving larger problems.**

Distributed Computing

1. Components **distributes** to different networked computers.
2. Communication through a unified **messaging mechanism.**
3. The components **work together in order** to achieve a common goal.

- **Benefits:**
  - Easier resource sharing
  - load balancing
  - Running programs on the most suitable computers.
- **Difference from parallel computing:**
  - **Parallel computing** → **shared** or **distributed memory.**
  - **Distributed computing** → each computer **has its own memory.**
  - both use **parallelism** to achieve higher performance.

Cloud Computing

- A new **infrastructure sharing model.**
- allows the sharing of super computational power over the Internet.

Grid Computing

- Special type of **distributed computing.**
- **Aggregates dispersed resources.**
- Appears as a **virtual supercomputer**.

---

3. Foster's Parallel Algorithm Design (4 Steps)

1. **Decomposition** → partitioning the problem into tasks.
2. **Communication** → connecting tasks to each other.
3. **Agglomeration** → reduce number of tasks to reduce overhead.
4. **Mapping** → assign tasks to processors.

Lecture 2: Terminology and Performance Metrics File

1.What is Parallel Computing

- Multiple processors cooperate concurrently to solve one problem.
- A parallel computer is a collection of processing elements that communicate and cooperate to solve large problems fast.

- **Terminologies:**
- **Core:** A single independent computing unit.
- **Multicore:** A processor with several cores accessing shared memory.
- **Task:** A part of computation executed in parallel.
- Finding enough parallelism is (one of the) critical steps for high performance (**Amdahl'slaw**).

---

2. Performance Metrics

Execution Time

- Time elapsed from start to end of execution.

Speedup

- Ratio of serial execution time to parallel execution time.
  - **Speedup = Ts / Tp**
  - **Ts =** Serial execution time. • **Tp =** Parallel execution time.

Amdahl's Law

- Quantifies the maximum achievable speedup.
- Considers the non-parallelizable portion.
  - **Formula:** $Speedp\ (S) = 1\ /((\ 1 - P) + P/N)$
  - **S** is the speedup of the parallelized program.
  - **P**: Parallelizable fraction. **N**: Number of processors.

Efficiency

- Ratio of speedup to the number of processors.
- **Efficiency= Speedup/P.**

Scalability

- examines how a parallel system performs when both the **problem size** and the **number of processors** grow simultaneously.
- **Weak scalability** remains execution time **constant or grows slowly.**

3.Challenges in Parallel Computing

Parallel programs always include both parallel and sequential parts.

Challenges include:

1) Sources of Overhead

- **Inter process interaction:** Time spent communicating between processors.
- **Idling:** due to load imbalance, synchronization, and serial components
- **Synchronization:** Managing shared resources, prevent data races, enable communication.
- **Parallelism rejection:** Difficulty in parallelizing some algorithms.

2) Memory System Challenges

- **Latency:** Time between memory request and data received.
- **Bandwidth:** Rate of data can pumped to the processor.
- Architectural innovations added multiple data paths to increase access to storage elements.

4.Approaches to Parallelism

Implicit Parallelism

- Parallelism is managed automatically by system or runtime.
- Programmer does not explicitly control threads.
- **Examples:** OpenMP, Java Fork-Join, Python concurrent.futures.
- **Advantages:**
  - Easier program design.
  - Programmer focuses on problem logic.
- **Disadvantages:**

- Programmer has less control.
- lower efficiency.
- Debugging may be difficult.

Explicit Parallelism

- Programmer explicitly controls parallelism.
- Programmer manually manages tasks, communication, and synchronization.
- **Examples:** MPI.
- **Advantages:**
  - Programmer has Full control.
  - High efficiency.
- **Disadvantages:**
  - Programming is difficult.
  - Extra effort in synchronization.

Goal: Utilize the Hardware, System, & Application Software to either:

- **Achieve Speedup.**
- **Solve problems requiring a large amount of memory.**

Lecture 3: OpenMP

1.Concurrency

- Two or more tasks executing in the same time interval.
- Concurrency does not necessarily mean execution at the same exact instant.
- Concurrent tasks can execute in:
  - single (context switching) or multiprocessing environment.

Layers of Concurrency

- **Instruction level**
  Multiple parts of a single instruction can be executed simultaneously.
- **Subroutine level**
  Functions assigned to different threads can execute simultaneously.
- **Object level**
  objects' methods in different threads or processes execute concurrently.
- **Application level**
  Two or more applications can cooperate to solve the same problem.

2.OpenMP

- **Application Programming Interface** (API) for **multi-threaded parallelization.** Uses **shared memory.**
- Consists of: • Functions
  - Source code directives
  - Environment variables

OpenMP Directives in C and C++ Based on #pragma.

Directive = directive name + clauses.

Clauses

- **if (expression)**: conditional parallelization.
- **num_threads(n)**: number of threads.
- **firstprivate(var)**: private and initialized.
- **private(var)**: local to each thread.
- **shared(var)**: shared across all threads.

3.Thread

- A thread is an instance of a function execution.
- Each thread has its own local stack.
- logical model of global (shared) memory + local (stack) memory.
- Threads are scheduled at runtime.

4.Work Sharing Constructs

- Divide execution of code among threads.
- Does not launch any new threads.
- Must be enclosed in a parallel region.
- Must be encountered by all threads.
- No implied barrier on entry.
- Implied barrier on exit unless nowait is used.

5.Section Directive

- Non-iterative work sharing construct.
- Divides work into separate, discrete sections.
- Each section is executed by a thread.
- Thread may execute more than one section.
- Used for functional parallelism.

6.Parallel For

- Parallelizes subsequent loop.
- Implicit barrier at the end.
- Clauses include:
  - private, firstprivate, lastprivate, schedule, reduction, nowait, ordered

---

7.Reduction Clause

- Combines local copies into one a single copy at the master when threads exit.
- Variables are implicitly private.
- Operators: +, *, -, &, |, ^, &&, ||

8.Scheduling

Static

- Iterations divided equally.
- Low overhead.

Dynamic

- default chunk size is 1
- Chunks assigned dynamically.
- Higher overhead than STATIC.

Guided

- default chunk size is 1
- assigns chunks automatically.
- Chunk size decreases exponentially.

Runtime

- Schedule defined using environment variable.

9.Nested Parallelism

- Enabled using OMP_NESTED.
- Allows parallel regions inside parallel regions.

10.Synchronization Constructs

barrier

- All threads wait until all reach the barrier.
- #pragma omp barrier

master

- Code executed only by master thread.
- No implicit barrier.
- #pragma omp master

single

- Only one thread executes the block.
- Other threads wait unless nowait.
- #pragma omp single

critical

- Mutual exclusion.
- Only one thread executes at a time.
- #pragma omp critical

atomic

- Single memory update.
- Mini critical section.
- #pragma omp atomic

ordered

- sequential loop execution.

Lecture4: MPI

1.What is MPI?

- MPI stands for Message Passing Interface.
- Used for parallel programming in distributed memory systems.
- Each process has its own local memory.
- Processes communicate by sending and receiving messages.

2.MPI Programming Model

- No shared memory between processes.
- Communication is explicit.
- Programmer controls data exchange.

3.Why MPI is used?

- Suitable for cluster systems.
- Supports large-scale parallelism.
- Highly scalable.

4.Communication in MPI

Point-to-Point Communication:

- One process sends data, Another process receives data.
- Examples: MPI_Send, MPI_Recv

5.Types of Collective Communication

Broadcast: One process sends data to all processes.

Scatter: Root process distributes different parts of data.

Gather: All processes send data to the root process.

Reduce: Combines values from all processes.

6.Global Reduction

- Combines values from all processes.
- Produces a single global result (sum, max, min).

Lecture 5: Parallel Algorithms Paradigms

1. Parallel Computing Platforms

Logical organization of a parallel program is divided into:

- Control structures
- Communication models

An explicitly parallel program must clearly define:

- Interaction between concurrent tasks.

2. Granularity

Granularity is a qualitative measure of the ratio of:

- Computation
- Communication

Fine-Grain Parallelism

- Small amount of computation between communication events
- Low computation-to-communication ratio
- Easier load balancing
- High communication overhead

Coarse-Grain Parallelism

- Large amount of computation between communication events
- High computation-to-communication ratio
- Harder load balancing

3. Task

- A task is a logically discrete unit of computation
- Executed by a processor
- Can be a program or part of a program

4. Flynn's Classical Taxonomy

Classification based on:

- Instruction stream
- Data stream

SISD (Single Instruction Single Data)

- Serial computer
- One instruction stream
- One data stream
- Example: traditional single-core machines

SIMD (Single Instruction Multiple Data)

- Same instruction executed on multiple data elements
- One control unit,One instruction, Multiple data.
- Example: vector processors, GPUs

MISD (Multiple Instruction Single Data)

- Multiple instruction streams
- Single data stream
- Rarely used
- Example: fault tolerance systems

MIMD (Multiple Instruction Multiple Data)

- Multiple instruction streams
- Multiple data streams
- Most common architecture
- Example: multicore systems, clusters
- Used with OpenMP and MPI

---

5. Communication Models

Communication models define how processors exchange data.

Shared Address Space (Shared Memory)

- Platforms that provide a shared data space
- multiprocessors

Message Passing Model

- Platforms that support messaging.
- multi-computer

---

6. Cache

What is Cache?

- A buffer between slow memory and high-speed registers.
- Copies a **cache line** from memory on access.
- Reduces average memory access time.

Single Processor Cache Operation

- **Read/write** → checks cache tags.
  - If found → cache hit → immediately reads or writes the data in the cache line.
  - Otherwise → cache miss → allocate a new entry → get data from memory → delay execution

hit rate

The proportion of accessesthat result in a cache hit. It is a measure of the effectiveness of the cache for a given program or algorithm

Cache Performance

- **Hit:** data in cache.
- **Miss:** data not in cache.
- **Hit rate (h)** = number of hits / total accesses
- **Miss rate (m)** = 1 – h

---

7. Cache Coherence Problem

- Multiprocessor systems have private caches.
- Multiple copies of data may exist.
- Updates cause **inconsistent data** (cache coherence problem).

8.Cache Coherence Solution

- **Coordinate access to shared memory.**
- **Write invalidate / write update protocol:**
  - After write, old copies cannot be used.
- **Hardware implementations**
  - Snoopy protocol.
  - Directory-based systems.
  - Distributed Directory

Lecture 6: Parallel Algorithms Paradigms 2

1. Snoopy Cache Protocol

- Based on a **broadcast bus**.
- All caches listen to invalidate and read requests.
- Caches snoop bus activities and perform coherence operations.
- Requires additional hardware.
- Simple but causes **bus contention**.

2. Directory-Based Protocol

- Snoopy systems broadcast to all processors.
- Directory systems send coherence requests only to required processors.
- Directory keeps track of:
  - Processors caching a block.
  - State of cache blocks.
- **Details**
  - Assume **K processors**:
    - Each memory block: K presence bits + 1 dirty bit.
    - Each cache block: 1 valid bit + 1 dirty bit.
  - Uses point-to-point messages.
  - Directory size increases with processors and memory size.

3. Distributed Directory Protocol

- Solves scalability problems.
- Memory is physically distributed among processors.
- Each processor maintains coherence for its own memory.
- Cache coherent, **non-uniform memory access**.

4. Sharing States

- **Uncached**: block not in any cache.
- **Shared**: cached by one or more processors (read-only).
- **Exclusive**:
  - Cached by exactly one processor.
  - Processor has written block.
  - Copy in memory is obsolete.